

Machine Learning Based Framework for Fast Resource Estimation of RTL Designs Targeting FPGAs

BENZHENG LI, XI ZHANG, HAILONG YOU, ZHONGDONG QI, and YUMING ZHANG, Xidian University School of Microelectronics

Field-programmable gate arrays (FPGAs) have grown to be an important platform for integrated circuit design and hardware emulation. However, with the dramatic increase in design scale, it has become a key challenge to partition very large scale integration into multi-FPGA systems. Fast estimation of FPGA on-chip resource usage for individual sub-circuit blocks early in the circuit design flow will provide an essential basis for reasonable circuit partition. It will also help FPGA designers to tune the circuits in hardware description language. In this article, we propose a framework for fast estimation of the on-chip resources consumed by register transfer level (RTL) designs with machine learning methods. We extensively collect RTL designs as a dataset, extract features from the result of a parser tool and analyze their roles, and train a targeted three-stage ensemble learning model. A $5,513\times$ speedup is achieved while having 27% relative absolute error. Although the effect is sufficient to support RTL circuit partition, we discuss how the estimation quality continues to be improved.

CCS Concepts: • **Hardware** → **Hardware description languages and compilation**; • **Computing methodologies** → **Supervised learning by regression**;

Additional Key Words and Phrases: Resource estimation, machine learning, FPGA, RTL design

ACM Reference format:

Benzheng Li, Xi Zhang, Hailong You, Zhongdong Qi, and Yuming Zhang. 2022. Machine Learning Based Framework for Fast Resource Estimation of RTL Designs Targeting FPGAs. *ACM Trans. Des. Autom. Electron. Syst.* 28, 2, Article 24 (December 2022), 16 pages.
<https://doi.org/10.1145/3555047>

1 INTRODUCTION

Modern **integrated circuit (IC)** designs for **field-programmable gate array (FPGA)** or **application-specific integrated circuit (ASIC)** targets are continuously becoming more complex, not only due to the increasing need for performance and functionality but also stricter requirements on size and power efficiency. This leads to an increasing workload on design and verification. In today's ASIC development, verification has become the most time-consuming part in the design flow. The **register transfer level (RTL)** verification requires testing the design in every possible scenario to meet the functional specification. It includes lint checking, formal model checking,

This research was supported by Key-Area Research and Development Program of Guangdong Province (2019B010140001) and the National 111 Center (B12026).

Authors' address: B. Li, X. Zhang, H. You (corresponding author), Z. Qi, and Y. Zhang, Xidian University School of Microelectronics, No. 2 South Taibai Road, Xi'an, Shaanxi, China, 710071; emails: lbz789@163.com, 290705951@qq.com, hlyou@mail.xidian.edu.cn, [zdzq, Zhangym]@xidian.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

1084-4309/2022/12-ART24 \$15.00

<https://doi.org/10.1145/3555047>

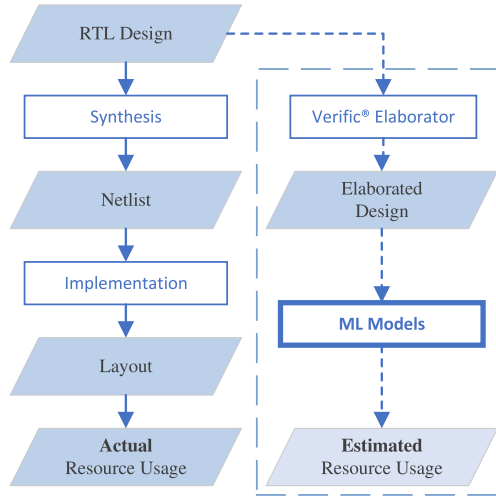


Fig. 1. A suite of ML models (right) is proposed to get resource usage without requiring traditional FPGA implementation flow (left).

logic simulation, transaction-based verification, and code coverage analysis. FPGA-based prototyping and emulation is a verification method that allows testing a design on hardware against real-time I/Os and feedback. The proposition of multiple FPGAs systems and virtual wires [19] greatly increases the design capacity.

FPGAs have fixed resources and area, and synthesis tools intend to utilize the fixed resources to achieve the user-defined performance goal. The FPGA hardware architectures contain not only basic programmable building blocks, including **look-up tables (LUTs)**, **flip-flops (FFs)**, and additional rounding logic, but also more complex on-chip blocks, such as **block random access memory (BRAM)**, **digital signal processing (DSP)** components, **carry chains (CARRY)**, and high-speed transceivers [32]. In the design synthesis flow, for a targeted RTL design, the amount of FPGA on-chip hardware resources it consumes is generally obtained when the synthesis and implementation flows are completed as the left half of Figure 1 shows. For larger-scale circuit designs, this flow often takes several hours. The verification and emulation based FPGAs of IC design face on the requirement of the estimation of the FPGA resources to enable the RTL-to-RTL partition flow. The estimates of the FPGA resources required provide the system designer with important feedback on area and cost, which is valuable during early design iterations. These scenarios will be elaborated in Section 2.

Almost 20 years ago, Enzler et al. [16], Nayak et al. [27], Bilavarn et al. [8], and Brandolese et al. [10] explored resource estimation for high-level language design. Meeuws et al. [26] propose models based on neural networks and **linear regression (LR)** to predict the utilization of slices on high-level languages. With today's perspective, these works are not excellent in terms of accuracy and speed. In recent years, with the gradual maturity of **high-level synthesis (HLS)** technology, resource estimation based on the resource information reported by HLS tools has gotten good results. The development of **machine learning (ML)** also makes it a good method for resource estimation. Dai et al. [13] construct a large C-to-FPGA dataset and identify relevant features from the HLS reports to train ML models to estimate the post-implementation metrics. Makrani et al. [25] propose the Pyramid framework to predict optimal timing as well as resource usage. This framework contains an ensemble model combining LR, neural network, **support vector machine (SVM)**, and random forest for estimation and achieves extremely high accuracy.

However, most RTL designs in practice are not generated by HLS but the designers. It is actually harder to estimate resource usage at a lower RTL than work at a higher level. During the HLS process, the tool has already performed an in-depth elaboration of the design. In addition, because they are the same in the Xilinx tool chain, the HLS tool is familiar with the behavior of subsequent synthesizers, so the resource reports given are relatively accurate. Bilavarn et al. [9] proposed a technique that estimates area, delay, and power through constructing several RTL designs and achieves 22% error in 20 cases. In 2008, Schumacher and Jha [30] of Xilinx proposed the usage of mathematical models on RTL design to estimate FPGA resource utilization. Makni et al. [24] propose a method that performs fast and rough logic synthesis, technology mapping, and other processes on RTL circuit designs to estimate resource usage. These three methods parse the circuit at a deeper level and use polynomials to fit the resource consumption. Ustun et al. [33] use **hardware description language (HDL)** descriptions as input and automatically extract features from logic synthesis, technology mapping, packing, placement, and routing stages. Its feature extraction stage does not satisfy the motivation to support circuit partition. With the complexity of FPGA structures and the development of their accompanying electronic design automation tools, the behavior of synthesizers has become more unpredictable. Simple polynomials are often difficult to produce stable and satisfactory fits in a variety of different types of circuit designs.

In this article, we propose an ML framework that is faster and more convenient than previous work. It can fast estimate the resource usage after the RTL design is implemented on the FPGA. Unlike much prior work, this framework is not limited to a specific circuit type or device platform. Its robustness is demonstrated in our large dataset. Its overall flow is shown in the right half of Figure 1. The main contributions of this work are as follows:

- (1) We collected a design dataset in Verilog language from a wide range of sources and trained a series of ML models to estimate its resource usage. A three-stage **ensemble learning (EL)** model is included, which eventually achieves a **relative absolute error (RAE)** with the value of 0.27 while running 5,513× faster.
- (2) Compared to the previous work, our framework removes the elaboration and hardware optimization phases of the process, thus greatly increasing the speed by at least 25×.
- (3) We analyze the impact of every feature and the behavior of the estimator to facilitate follow-ups. The optimization direction of the model is also discussed.

The rest of the article is organized as follows. Section 2 explains our motivation for doing this work. Section 3 describes the data collection and feature extraction process. The ML framework and details are presented in Section 4. Section 5 shows our experimental results and related analysis. We present our conclusion and plans for future work in Section 6.

2 MOTIVATION

As mentioned in the previous section, logic emulation of multi-FPGA-based systems has become the dominant approach to solving the design scale explosion problem. When designers want to use a multi-FPGA system, a reasonable partitioning scheme is especially important, which will affect the system performance, area, wiring success rate, and other factors [23]. To ensure proper resource allocation after partitioning, a fast and high-quality partitioning scheme requires an early estimate of multiple resource usage. Hung and Sun [22] mention that the current FPGA system circuit partition contains two flows: the netlist-level partition and RTL partition. There is a debate between these two schools of thought regarding where technology mapping should happen. As Figure 2 shows, netlist-level partition first performs synthesis and then partitions the netlist, whereas RTL partitions partition the RTL design first and then each partition performs synthesis separately.

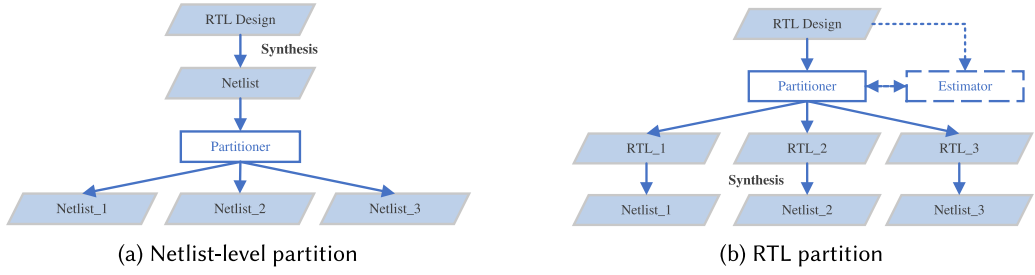


Fig. 2. Two partition flows for hardware emulation.

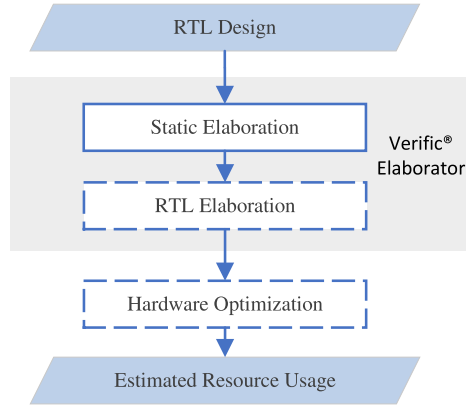


Fig. 3. Compared with the estimator proposed by Schumacher and Jha [30], our framework avoids the steps in the dotted box.

Currently, netlist-level partition dominates usage and research due to its ability to be more easily abstracted as a hypergraph. However, RTL partition has many advantages, including the ability to run synthesis in parallel, the circuit boundary information to guide the synthesis, and the existence of some high-level information to guide the partition. The bottleneck preventing its widespread use is that the absence of synthesis leads to the most important basis of partitioning: resource utilization information missing. Therefore, how to quickly estimate the resources of the design at the RTL without logical synthesis has become a key problem to be solved in RTL partition.

Xilinx proposed a rapid RTL resource estimation model in 2008 [30], which proposes a netlist-based resource calculation method. Fast logic synthesis is achieved by fast elaborate and hardware optimization of the input RTL design to obtain the resource estimation results required for the RTL design as Figure 3 shows. Although the speed is improved compared with the fully synthesized flow, it still requires a lot of memory and runtime for mega designs. Since multiple iterations are required to find reasonable hypergraph node boundaries, it is necessary to estimate the resources at different levels of the design several times. So, it does not meet the needs of fast estimation for RTL-partition-oriented processes. Our work focuses on the fast estimation of FPGA resource usage for the RTL design. The proposed ML framework in this article does not require an elaborate and hardware optimization step, so it is faster and more convenient than previous work.

Here, we will illustrate the use of estimators in circuit partition with an example of a practical application scenario. In the initial stages of RTL partitioning, the RTL design needs to be mapped into a hypergraph structure. Too many hypergraph nodes will increase the time consumption of the partitioner, whereas too few nodes will deteriorate the quality of the partitioning. An ideal

Table 1. Post-Implementation Resource Usages of Dataset Designs

Resource	#LUT	#FF	#BRAM	#DSP	#CARRY
Max	671,472	235,930	347	207	5,492
Min	0	0	0	0	0
Mean	3,298	1,275	1.78	0.84	37.59

solution requires that the designs in each node contain a similar and reasonable amount of resources. First, we calculate the average resources based on FPGA on-chip resources and the number of nodes to be generated. This is set as a threshold value that means the upper limit of the resource value. Next, we will estimate the resources of all underlying modules. If their resources do not exceed the threshold, then we continue to estimate their parent modules. This process continues until the module's resources just exceed the threshold, and the instances at this level are treated as nodes. The resource consumption of these nodes will also be recorded in the node information as an important basis for partitioning. In addition, we need to estimate the resources on each FPGA after the partitioning is done to check if it is over the limit. An actual RTL circuit partition is performed to prove the validity of the flow in Section 5.2.

This application scenario does not require an estimator with very high accuracy, because a circuit partition often leaves a resource margin of 30% to 50% to avoid implementation failures such as routing congestion. In addition, the hypergraph produced by the RTL file transformation before partition often has millions of nodes, and the small number of nodes with large resource deviations often has only a negligible impact. At the same time, fast resource estimation during the RTL phase can help designers quickly capture the impact of decisions and guide them to modify designs.

With the development of ML in the field of electronic design automation, it captures a wide range of factors affecting results in a comprehensive and accurate manner. ML algorithms focus on extracting high-level features or patterns that can be reused in other related or similar situations, avoiding repeated complicated analysis [21]. We believe that it is an extremely suitable means to accomplish the resource forecasting task when the dataset size is sufficient.

3 DATA PROCESSING AND ANALYSIS

3.1 Data Collection

To accomplish RTL resource estimation with ML, we constructed a dataset that contains 715 RTL designs in Verilog language. To guarantee the robustness and training quality of the estimator, the source of designs in our dataset cover a very wide range. They come from the well-known FPGA open-source IP core website OpenCores [1], RTL projects in the open-source community GitHub [3], and open-sourced industrial designs. We also introduced the well-known HLS benchmarks CHStone [18] and MachSuite [29] and generated the corresponding RTL designs by HLS using the Vitis 2021 HLS tool [5]. These designs cover a wide range of applications, such as image processing, neural networks, DSP, codecs, and CPU cores. The different data sources, application types, and design scales make the HDL code features vary greatly from one design to another, which poses a great challenge for training and estimation. Table 1 shows the overview for actual usage of each kind of resource in the dataset.

3.2 Design Elaboration and Synthesis

Further, we extracted possible features for these designs to make ML feasible. Verific Design Automation [2] builds HDL parser platforms that are leveraged to analyze the designs. Verific's parser platform supports both static elaboration and RTL elaboration. Static elaboration will elaborate

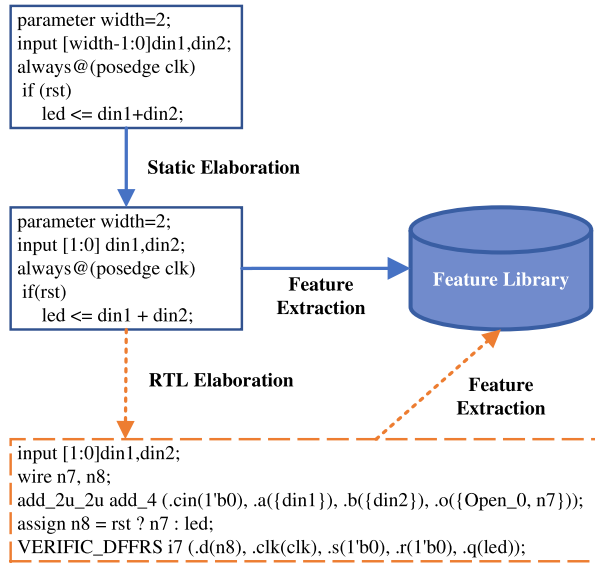


Fig. 4. Verific tool elaborate design in two levels. Unlike previous works, we extract features only from the static elaborating result as shown in the blue solid line.

the entire language and specifically bind instances to modules, unroll generate statements, and so forth. After a Verilog design has been parsed, the design must have the modules being instantiated linked to modules being defined, the parameters propagated among the various modules, and hierarchical references resolved. This phase is called *static elaboration*. The role of static elaboration includes identifying top-level modules and tree of instantiations under each of them, processing generate statements, flattening instance arrays, replacing parameter values with constant values, and so on. The result after static elaboration is an RTL design stored in the parse tree data structure of Verific. RTL elaboration generates a netlist on generic gates such as DFF, AND, OR, and MUX, which is appropriate for applications such as logic synthesis and formal verification. It provides specific information on the use of logic gates and FFs, which are helpful for evaluating the utilization of LUTs and FFs. However, the CPU time consuming of RTL elaboration is a multiple of static elaboration, which is shown in Section 5.

Unlike any previous work, we discard RTL elaboration and extract features only from the results after static elaboration as Figure 4 shows. The three boxes in Figure 4 from top to bottom show the original code, the code after static elaboration, and the code after RTL elaboration. In this way, we get ultimate operating speed to meet the challenges of ultra-large-scale designs. Static elaboration takes only a few seconds to run on the largest design in our dataset.

We use Vivado [6] to perform implementations of the designs to get their resource usage, i.e., the labels of training data. Vivado's synthesis tool has a variety of customizable configuration parameters, as well as some synthesis modes that are already pre-defined, such as area optimization or runtime optimization. In this work, we operate in the default mode. Devices need to be decided before synthesis as well. We tried several different FPGA boards and noticed that the device barely impacts the resource usage when board resources are sufficient. As one of the most advanced devices, Virtex UltraScale is used in this work. For adapting this estimator framework to other specific modes or devices, changing the synthesis options and regenerating the labels will suffice. Although the actual resource usage will not be available until implementation finished, the difference between the resource usage reported after synthesis and actual resource usage is negligibly

small. Thus, we use the resource usage reported by Vivado after synthesis as the label because it is very close to the final result and with much lower time consumption.

3.3 Feature Extraction

Verific stores the results of static elaboration in its parse tree data structure, which includes the type and member of every statement or expression. We extract features from this data structure. In fact, this is equivalent to extracting features directly from the elaborated Verilog code, just because the former is more convenient to achieve. If there is a loop statement in the design, we simply parse it to get the number of loops possible. Since the for-loop in the circuit design is usually used only to enumerate the indexes of the registers, this parsing is effective.

First, we extract a large number of features that are relevant and possibly correlated with our estimation task. Next, to overcome the effect of collinearity, we calculate Pearson's correlation coefficients of each pair of features, and we keep only one from the group of features with large correlation. We also calculated the correlation coefficients between each feature and each target resource, and removed features that had little correlation with any kinds of resource. After the initial screening, the feature extraction left 46 features to reflect the actual usage of the resources.

We tried to use all of these features to train the model and estimate, but the results were not satisfactory. It may be because the impact of many important features is diluted. To solve this problem, we pick some potentially high correlation features specifically for each estimation task, based on our experience in IC design. Many experiments were then done to help decide which features to remove or add. All resources ultimately involved in the training are listed in Table 2. The value in the table means that this feature is used as input and removing it will worsen the RAE of the estimation result. For example, removing "total bit width of assign" from the feature set will increase the RAE of LUT and FF estimation by 0.006 and 0.002, respectively. In addition, features that passed initial screening but were rejected in the course of the experiment are listed in Table 3.

The required features for each estimation task are analyzed as follows.

BRAM. "Number of large-scale memorys with only one write port" is critical to the estimation of BRAMs. Generally, large-scale memory in synchronous circuits with only one write port will be synthesized as BRAMs. In other cases, memory is easily synthesized as LUTRAMs (special LUTs) for performance and functional considerations.

DSP and CARRY. "Total bit width of plus operation between two regs" is critical to the estimation of DSPs. Generally, multiplication operations between registers of large bit width are most easily synthesized into DSP, whereas small bit width or register-to-parameter operations are sometimes implemented using CARRY. So if we can find a good way to distinguish the scale of the operations, the results can be improved. In fact, division, remainder, and power also affect the DSP, but since these features are too sparse in our dataset, they are removed temporarily to improve the training accuracy.

LUT. As the most common resource in FPGAs, almost all features affect the use of LUTs. It is worth noting that features that are highly correlated to both BRAM and DSP are still important when estimating LUTs. Since the usage of DSP and BRAM resources can greatly affect the number of LUTs. But our estimator uses the number of DSPs, MEMs, and CARRYs as input, as described in Section 4.2.2. Their associated features would no longer need to be present in the LUT estimation.

FF. Similar to the case of LUT estimating, features that correlate to BRAMs, DSPs, and CARRYs are also important, and they would no longer need to be present in the estimation.

Table 2. Impact of Features on Estimation Tasks

Feature description	LUT	FF	BRAM	DSP	CARRY
Number of modules	0.003	×	×	×	×
Number of defined regs	×	×	0.050	×	×
Total bit width of defined regs	0.009	0.064	0.002	×	×
Number of memories	×	×	0.005	×	×
Number of large-scale memories with only one write port	×	×	0.573	×	×
Number of always statements	0.006	0.002	0.008	×	×
Number of case statements	0.023	×	×	×	×
Number of if statements	0.004	×	×	×	×
Total bit width of assign	0.006	0.002	×	×	×
Total bit width of blocking assign	0.005	×	×	×	×
Total bit width of nonblocking assign	0.020	0.055	×	×	×
Total bit width of equivalence judgment	0.008	×	0.016	×	×
Total bit width of plus operation between reg and const	×	×	×	×	0.014
Total bit width of plus operation between two regs	×	×	×	0.031	0.023
Total bit width of subtract operation between reg and const	×	×	×	×	0.004
Total bit width of subtract operation between two regs	×	×	×	0.006	×
Total bit width of plus operation between two regs	×	×	×	0.585	×
Total bit width of bit operation	0.022	×	×	×	0.004
Total bit width of logic operation	×	×	0.002	×	0.001
Number of shift operation	×	×	×	×	0.003
Total bit width of greater/less than judgment	0.011	×	×	×	×
Total bit width of defined wires	×	0.007	×	×	×
Average bit width of multiplication operations	×	×	×	0.047	×

The mark × means that this feature is not used in the corresponding task.

Table 3. Features That Passed the Initial Screening But Were Rejected in the Course of the Experiment

Total bit width of input	Number of small-scale memories
Total bit width of output	Number of assign statements
Total bit width of multiplication operation between reg and const	Number of blocking assign
Total bit width of division operation between reg and const	Number of nonblocking assign
Total bit width of division operation between two regs	Number of conditional statements
Total bit width of remainder operation between reg and const	Number of plus operations
Total bit width of remainder operation between two regs	Number of subtract operations
Total number of simple loop execution	Number of multiplication operations
Total number of items in case statements	Number of division operations
Number of functions	Number of remainder operations
Number of loop statements	Number of power operations

4 ESTIMATION MODELS

We use regression models to estimate the usage of LUTs, FFs, BRAMs, DSPs, and CARRYs. Regression is a well-known statistical supervised learning technique that gives the relationship between feature vectors and labels. In this work, the input of our model is a given vector $\vec{x} = [x_1, x_2, \dots, x_n]^T$, and the output is a five-dimensional vector $\vec{y} = [y_{LUT}, y_{FF}, y_{BRAM}, y_{DSP}, y_{CARRY}]^T$. We can either use a multitask estimation with a one-time output $\vec{y}$ or use five single-task estimations to obtain the five-dimensional values of $\vec{y}$ separately. As mentioned in the work of Dai et al. [13], the single-task model gives better results in FPGA resource estimation, and the single-task regression model is also adopted by us. There are various

ML models that can accomplish the task. In this work, we tried LR, **random forest regression (RFR)** [14], **support vector regression (SVR)** [15], **artificial neural network (ANN)** [20], **extreme gradient boosting (XGB)** [12], and a customized two-stage EL to estimate four kinds of resources, respectively.

4.1 Traditional Regression Models

Linear regression. LR is a type of regression analysis that assumes linear correlation between target values and features. It is simple and fast and is often used as a first attempt to solve regression problems. Let $\hat{y}_i = w_i x_i$ and we find each component of the vector $\vec{w}$ such that the value of the loss function $\sum (\hat{y}_i - y_i)^2$ is minimized.

Random forest regressor. RFR is a meta-estimator that fits the values of a regression decision tree on a subsample of a set of datasets. Increasing the number and depth of trees in the forest can increase the accuracy of the regression. As the number of decision points for optimal partitioning increases, the size of the ML model becomes more extensive and requires more adjustment time.

Support vector regression. SVM solves binary classification problems by formulating them as convex optimization problems. As in classification, SVR is characterized by the use of kernels, sparse solution, and Vapnik-Chervonenkis control of the margin and the number of support vectors. Although less popular than SVM, SVR has been proven to be an effective tool in real-value function estimation [7]. One of the main advantages of SVR is that its computational complexity does not depend on the dimensionality of the input space. It is the fastest nonlinear model used in this work.

Artificial neural network. An ANN consists of an input layer, a series of hidden layers, and an output layer. Each hidden layer contains a set of nodes that are arranged in weighted connections in the layers. Each layer uses the output of the previous layer as input. The activation function adds nonlinearity to the model. According to Shahshahani and Bhatia [31] and Dai et al. [13], because of the wide range of outputs of our model (especially for LUTs and FFs) and the experimental results that demonstrate it, using ReLU as the activation function instead of sigmoid or tanh will achieve better results. For our estimation problem, because of the small number of features compared to large-scale neural networks and the small data sample size of the training set, we found in our experiments that using a deep network structure would easily produce overfitting problems. Therefore, our ANN structure has only a few fully connected hidden layers. Unlike LR and random forest, ANN can represent more complex nonlinear structures and also requires larger sample size and longer training time, but can achieve better estimation results.

Extreme gradient boosting. Gradient boosting regression is an ML technique that gives a prediction model in the form of an ensemble of weak prediction models, which are typically decision trees [17]. XGBoost is an optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable. It implements ML algorithms under the gradient boosting framework. XGBoost provides a parallel tree boosting (also known as GBDT and GBM) that solves many data science problems in a fast and accurate way. Experimental results demonstrate that among all of the preceding models, XGB has the best performance.

4.2 Customized Estimation Models

4.2.1 Ensemble Learning. EL accomplishes learning tasks by building and merging multiple learners. In this way, the accuracy and stability of the model is improved. In our model, the learners will be weighted averaged as Figure 5 shows. To make more accurate learners account for more

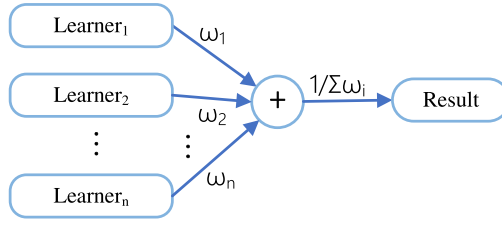


Fig. 5. EL schematic.

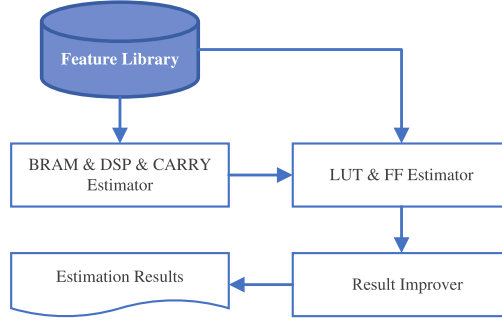


Fig. 6. Overflow of our multistage regression estimator.

parts, the weight is defined as follows:

$$\omega_i = \alpha_i / \epsilon_i,$$

where the ϵ_i means the estimation error on the validation set. We tried all combinations of the five learning models and finally picked RFR, ANN, and XGB for application. Based on the experimental performance of the learner alone, we assign $\alpha_{ANN} = 1$, $\alpha_{RFR} = 2$, and $\alpha_{XGB} = 4$. In this work, EL is applied to all estimation tasks to improve the accuracy and stability of the model.

4.2.2 Multistage Model. To further improve the estimation accuracy, we use a three-stage regression model to accomplish the task as Figure 6 shows. On the one hand, the usage of DSP, BRAM, and CARRY resources can greatly affect the number of LUTs and FFs. For example, the number of LUTs is reduced by several hundred with the use of one DSP compared to the integrated approach without using any DSP. On the other hand, the number of correlation features that determine the DSPs, BRAMs, and CARRYs is relatively small, and the fluctuations and errors in estimation are relatively small as well. Therefore, the utilization of DSPs, BRAMs, and CARRYs are estimated in the first stage.

Before the start of the second stage, the estimated DSP, BRAM, and CARRY results are added to the input vector $\vec{x}$ as three new feature dimensions. Note that the data in the training set are still estimated values when added to $\vec{x}$, even though the actual resource usage is known. The reason is that keeping the training and test sets consistent can result in better model performance, as demonstrated by the experiment. After that, two models are trained on the new input vectors to estimate the LUT and FF usage, respectively.

In the third stage, the results are refined based on some analysis of IC-related knowledge. In this work, only simple refinements are considered:

- (1) The number of resources is non-negative, so we correct the negative result to 0.

- (2) If there is no always statement in the Verilog code, then the circuit is generally a combinational logic circuit, and there should not be any FFs. Therefore, if the feature value of the number of always statements is 0, the number of FFs is set as 0.

If more features are available and more patterns are uncovered in the future, they could be added in the third phase of the framework. Compared with the traditional ANN, this model structure can deliver more accurate results.

5 EXPERIMENTAL RESULTS

We use the model described in Section 4. The experiments are completed in the Python environment using the Keras [4] and Scikit-learn [28] libraries. Designs in our dataset are synthesized with Xilinx Vivado 2021.2 targeting Virtex UltraScale. All operation are completed with the default tool options. The experiments are completed on the platform Intel Xeon 6238 CPU at 2.10 GHz. Experiments are conducted on entire designs in our dataset and an actual RTL circuit partition separately to help evaluate the performance of the estimator.

5.1 Evaluation on Entire Designs

We choose to use RAE as one of the evaluation indicators, like in the work of Dai et al. [13]. It is defined as follows:

$$RAE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (\bar{y} - y_i)^2}}.$$

The $\hat{y}$ is the estimated value and y is the actual values in the testing set. $\bar{y}$ denotes the mean value of y . However, the RAE indicator is not very compatible with this problem, because it does not include the consideration of case size in the denominator and squares the error. This leads to its results being very strongly influenced by large-scale cases, and thus the estimation performance of small-scale cases is almost ignored. We therefore propose another evaluation metric called **relative normalized error (RNE)** to jointly measure the error. It is defined as follows:

$$RNE = \frac{1}{n} \sum_{i=1}^n \frac{|\hat{y}_i - y_i|}{\hat{y}_i + y_i + 1}.$$

For a particular case, the RNE is 0 when the estimation is perfectly accurate and 1 when the estimation deviation is infinite. The “+1” in the denominator is to prevent division by 0. This prevents a single exception case from having an excessive impact on the overall result while balancing the impact of cases of various sizes.

A fivefold cross validation is performed on the dataset: all data are randomly divided into five equal-sized groups. One group of the data is selected as the test data each time, and the training/validation is performed with the remaining data. In each iteration, we randomly select 75% of the training/validation set for training and 25% for validation. The validation set is used to tune the parameters to get a better model, whereas the test set is guaranteed not to be involved in any training other than the final evaluation. The training is terminated at a certain number of epochs. Finally, RAEs and RNEs of the estimation results on the test set are taken to evaluate the whole model.

The means and standard deviations of RAEs and RNEs generated by cross validation for LR, RFR, SVR, XGB, ANN, EL, and **single-stage EL (SSEL)** are shown at Tables 4 and 5. In all experiments but SSEL, we used the three-stage method in Section 4.2.2 to get the best results. The results show that XGB achieves the best performance at RAE metrics, whereas EL achieves the best RNE value. This shows that XGB is good at handling large-scale designs and EL brings better comprehensive performance. At the same time, the smaller standard deviation than XGB indicates that EL has

Table 4. Resource Estimation Result

Learner	LUT	FF	BRAM	DSP	CARRY
LR	0.440 (+/- 0.372)	0.246 (+/- 0.193)	0.320 (+/- 0.462)	0.715 (+/- 0.198)	0.434 (+/- 0.287)
RFR	0.341 (+/- 0.151)	0.362 (+/- 0.149)	0.304 (+/- 0.117)	0.484 (+/- 0.220)	0.341 (+/- 0.175)
SVR	0.441 (+/- 0.473)	0.477 (+/- 0.448)	0.477 (+/- 0.453)	0.531 (+/- 0.260)	0.573 (+/- 0.369)
XGB	0.199 (+/- 0.222)	0.189 (+/- 0.207)	0.209 (+/- 0.232)	0.454 (+/- 0.218)	0.305 (+/- 0.333)
ANN	0.383 (+/- 0.273)	0.487 (+/- 0.407)	0.736 (+/- 0.946)	0.611 (+/- 0.146)	0.417 (+/- 0.246)
EL	0.217 (+/- 0.180)	0.231 (+/- 0.187)	0.228 (+/- 0.247)	0.427 (+/- 0.219)	0.285 (+/- 0.263)
SSEL	0.228 (+/- 0.174)	0.281 (+/- 0.133)	0.228 (+/- 0.247)	0.427 (+/- 0.219)	0.285 (+/- 0.263)

The average RAE values of the five validation results are shown, with the standard deviation of the RAE values in parentheses.

Table 5. Resource Estimation Result

Learner	LUT	FF	BRAM	DSP	CARRY
LR	0.472 (+/- 0.048)	0.394 (+/- 0.075)	0.032 (+/- 0.006)	0.034 (+/- 0.014)	0.437 (+/- 0.120)
RFR	0.369 (+/- 0.020)	0.277 (+/- 0.031)	0.031 (+/- 0.003)	0.022 (+/- 0.005)	0.206 (+/- 0.022)
SVR	0.567 (+/- 0.004)	0.700 (+/- 0.022)	0.060 (+/- 0.035)	0.027 (+/- 0.009)	0.227 (+/- 0.016)
XGB	0.382 (+/- 0.033)	0.315 (+/- 0.038)	0.029 (+/- 0.003)	0.021 (+/- 0.005)	0.247 (+/- 0.023)
ANN	0.449 (+/- 0.013)	0.602 (+/- 0.065)	0.040 (+/- 0.004)	0.028 (+/- 0.009)	0.189 (+/- 0.010)
EL	0.359 (+/- 0.027)	0.279 (+/- 0.033)	0.028 (+/- 0.004)	0.021 (+/- 0.004)	0.234 (+/- 0.020)
SSEL	0.376 (+/- 0.021)	0.305 (+/- 0.041)	0.028 (+/- 0.004)	0.021 (+/- 0.004)	0.234 (+/- 0.020)

The average RNE values of the five validation results are shown, with the standard deviation of the RNE values in parentheses.

Table 6. Best RAE and RNE Values of the Data Group in Five Validations for Each Resource

Resource	LUT	FF	BRAM	DSP	CARRY
Best RAE	0.021	0.040	0.034	0.286	0.102
Best RNE	0.351	0.253	0.023	0.016	0.161

stronger stability. In SSEL, we use the features used for estimation of BRAMs, DSPs, and CARRYs, rather than their results, as input features for LUT and FF estimation. The comparison of EL and SSEL shows that phased estimation improves accuracy.

The best results obtained for each resource in the five groups of test data are shown in Table 6. We have taken an example at each magnitude to show the actual estimation as Table 7 shows. For LUT and FF estimation, the cases with errors less than 20% are 18.2% and 36.7%, respectively. In summary, the multistage estimator can achieve better results compared to the general regression model.

The scatter plot of estimated versus actual values for all data is shown in Figure 7. The points below the line $y = x$ indicate that the estimated value is smaller than the actual value, whereas points above indicate that it is larger. For large-scale design, the LUT has many outliers. Analysis reveals that these points are often cases that actually use more DSP and BRAM resources. Regarding RAM, there are some points that are very much on the low side. This is usually because the synthesizer accidentally maps a lot of small-scale memory into BRAM. The introduction of DSP and BRAM brings a big challenge in the synthesis result estimation. In fact, the estimation effect of RTL designs produced by the HLS tool tends to be poor. This is probably because their coding style is quite different from most cases done manually. In this regard, we recommend training these designs individually with a sufficient sample size.

Table 7. Estimated FPGA Resources and Actual Post-Map Results for Nine Designs

Design	#LUT		#FF		#BRAM		#DSP		#CARRY	
	Actual	Estimated	Actual	Estimated	Actual	Estimated	Actual	Estimated	Actual	Estimated
openc910	669,902	552,352	235,730	204,997	347	329	0	0	5,486	4,954
nova	22,472	11,243	6,462	10,652	1	4	4	4	345	488
sin	10,634	11,029	5,591	8,819	0	0	43	25	431	371
jpegcore	5,492	4,352	3,043	4,540	2	2	31	25	372	235
sha1	1,001	569	1,001	499	0	0	0	0	28	25
hight	544	533	134	305	0	0	0	0	2	9
account	100	168	88	47	0	0	0	0	8	10
gng_lzd	51	43	0	0	0	0	0	0	0	0
divodd	10	16	12	17	0	0	0	0	0	1

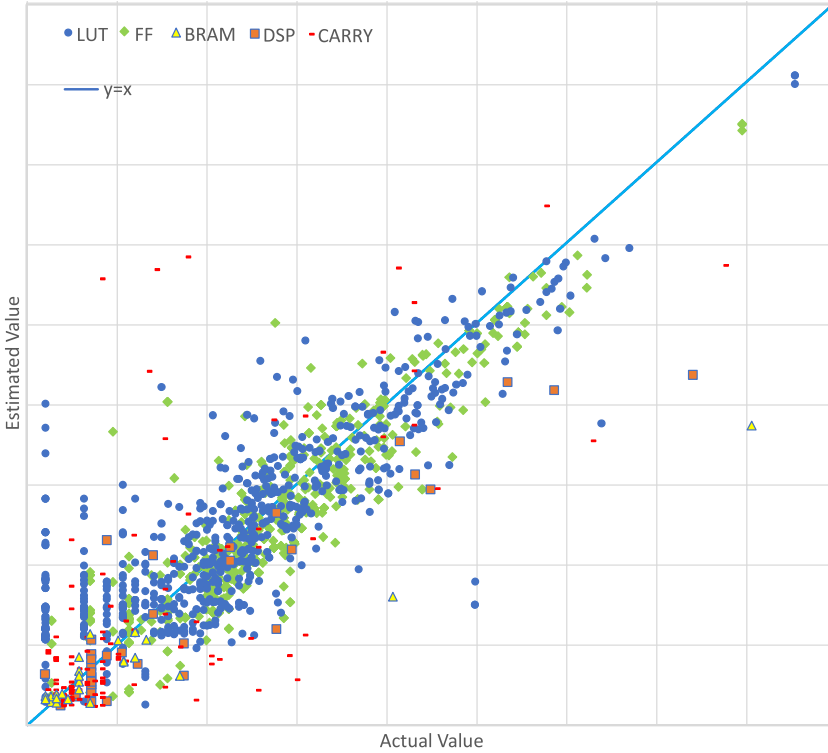


Fig. 7. Scatter plot of estimated versus actual values on five kinds of resource.

The runtime of the cases in Table 7 is shown in Table 8. The maximum number of CPU threads in Vivado's synthesis is set to 64 to accelerate large-scale cases. The time required for feature extraction after parsing is almost negligible. On the one hand, compared with our framework, the estimator in the work of Schumacher and Jha [30] needs RTL elaboration and a hardware optimization process. As a result, this work increases the speed by at least 25 $\times$. On the other hand, compared with traditional synthesis estimation, our estimator achieves an average of 5,513 $\times$ runtime speedup. Vivado takes 16 hours to run all of the cases, whereas ML takes 56 seconds. Even the smallest design of Vivado took 23 seconds to complete the synthesis flow, whereas the longest time taken by our estimator was only 16 seconds.

Table 8. Runtime of Static Elaboration, RTL Elaboration, and Vivado Synthesis

Design	Static Elaboration(s)	RTL Elaboration(s)	Vivado Synthesis(s)
openc910	15.67	90	1,354
nova	0.31	7.6	141
sin	0.12	7.9	209
jpegcore	0.058	5.5	59
sha1	0.0046	0.083	32
hight	0.011	0.043	31
account	0.0024	0.015	28
gng_lzd	0.0037	0.012	25
divodd	0.0017	0.0035	25
Ave. ratio	1	25	5,513

5.2 Evaluation on RTL Circuit Partition

This section shows how the estimator works in the actual RTL circuit partitioning. OpenC910 [11] is an open-sourced large-scale industry leading 64-bit high performance embedded RISC-V processor, and it was picked as the design to be processed. For the validity of the experiment, we trained the estimator with all of the samples from our dataset except openC910.

According to the process described in Section 2, extracting a hypergraph from the RTL design is the first step. To reserve margin to complete routing, we chose 50% as the FPGA available resource threshold based on experience. We set the target node number of the hypergraph to 500. This design contains 15,700 instances, and 509 nodes with estimated resource utilization information were finally constructed in the hypergraph.

Next, the hypergraph partitioner divided the nodes into four FPGAs. Because of the multidimensional resource constraint, we use the algorithm proposed by Li et al. [23] to complete the hypergraph partitioning. After unfolding the circuit in the nodes into RTL code, the partition is complete. We counted the sum of estimated resources of the nodes in the hypergraph on each FPGA and compared them with the actual utilization of the circuit on each FPGA after partition. Results on four FPGAs are shown in Table 9. The most strained resource is the LUT on FPGA-1, which uses 59% of the on-chip resources. However, it is only 9% more than the setup value (50%), which can ensure that it will not affect the subsequent operations such as interface insertion and routing. The elapsed time for resource prediction and partitioning is 4.1 seconds and 0.2 seconds, respectively. In summary, our estimator can help complete the RTL partition operation.

6 CONCLUSION

In this article, we proposed an ML framework to solve the problem of fast resource estimation for RTL designs. The solution of this problem is beneficial for hardware emulation and circuit design applied to FPGA platforms. At the beginning, we collected a dataset of RTL designs in Verilog language from many sources. Then static elaboration were performed using the Verific tool. Features were extracted from elaborated codes, and the critical features were analyzed. Several different estimation models were then tried, including a targeted proposed three-stage EL model. The experimental results showed that this model significantly improves the estimation accuracy while having a very fast running speed. The estimator was applied to an actual RTL circuit partition, and its effectiveness was proved. We also analyzed the factors that may affect the accuracy of the results.

Table 9. FPGA On-Chip Resource Situation

	FPGA ID	#LUT	#FF	#BRAM	#DSP	#CARRY
Actual	1	210,330	100,036	262	0	924
	2	178,554	52,731	18	0	957
	3	161,722	45,904	34	0	1,176
	4	154,296	41,319	34	0	2,434
Estimated	1	179,011	95,725	100	0	634
	2	177,545	84,605	14	0	728
	3	177,469	75,004	28	0	386
	4	148,384	75,568	26	0	1,436
Available	–	358,080	716,160	1,260	600	44,760

The first four rows show the actual resource usage after implementation, whereas the next four rows show the sum of the estimated resources of hypergraph nodes. The last row shows the available resources of the FPGA device.

The framework proposed in this work is widely applicable, although preliminary and rough. There is still much work to be done in the future to further improve the estimating results. First, close to 700 data is still insufficient for ML, and collecting more test data will make model training different. Second, we believe that there are still many circuit features waiting to be discovered, which can be guided by careful analysis of the synthesizer's behavior, and we believe that experienced circuit designers can do a great job. Our future work is not only to continue optimizing this framework but more importantly to apply it to the RTL partitioner to actually improve the performance of emulation. How to fully utilize its extremely fast speed to guide the partition to achieve resource balance is the focus of research.

REFERENCES

- [1] OpenCores. 1999. OpenCores Home Page. Retrieved October 6, 2022 from <https://www.opencores.org>.
- [2] Verific. 2000. Verific Design Automation. Retrieved October 6, 2022 from <https://www.verific.com/>.
- [3] GitHub. 2008. GitHub Home Page. Retrieved October 6, 2022 from <https://www.github.com>.
- [4] Keras. 2015. Keras Home Page. Retrieved October 6, 2022 from <https://keras.io>.
- [5] Xilinx. 2021. UG1399—Vitis High-Level Synthesis User Guide. Available at <https://www.xilinx.com>.
- [6] Xilinx. 2021. Vivado ML Standard Edition. Available at <https://www.xilinx.com>.
- [7] Mariette Awad and Rahul Khanna. 2015. Support vector regression. In *Efficient Learning Machines*. Springer, 67–80.
- [8] Sébastien Bilavarn, Guy Gogniat, Jean Luc Philippe, and Lilian Bossuet. 2003. Fast prototyping of reconfigurable architectures from a C program. In *Proceedings of the 2003 International Symposium on Circuits and Systems (ISCAS'03)*, Vol. 5. IEEE, Los Alamitos, CA.
- [9] Sebastien Bilavarn, Guy Gogniat, J.-L. Philippe, and Lilian Bossuet. 2006. Design space pruning through early estimations of area/delay tradeoffs for FPGA implementations. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25, 10 (2006), 1950–1968.
- [10] C. Brandolese, W. Fornaciari, and F. Salice. 2004. An area estimation methodology for FPGA based designs at system-level. In *Proceedings of the 41st Design Automation Conference*.
- [11] Chen Chen, Xiaoyan Xiang, Chang Liu, Yunhai Shang, Ren Guo, Dongqi Liu, Yimin Lu, et al. 2020. Xuantie-910: A commercial multi-core 12-stage pipeline out-of-order 64-bit high performance RISC-V processor with vector extension: Industrial product. In *Proceedings of the 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA'20)*. IEEE, Los Alamitos, CA, 52–64.
- [12] Tianqi Chen, Tong He, Michael Benesty, Vadim Khotilovich, Yuan Tang, Hyunsu Cho, Kailong Chen, et al. 2015. XGBoost: Extreme gradient boosting. *R Package Version 0.4-2* 1, 4 (2015), 1–4.
- [13] S. Dai, Y. Zhou, H. Zhang, E. Ustun, and Z. Zhang. 2018. Fast and accurate estimation of quality of results in high-level synthesis with machine learning. In *Proceedings of the 2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM'18)*.
- [14] T. G. Dietterich. 2000. An experimental comparison of three methods for constructing ensembles of decision trees: Bagging, boosting, and randomization. *Machine Learning* 40, 2 (2000), 139–157.

- [15] Harris Drucker, Christopher J. Burges, Linda Kaufman, Alex Smola, and Vladimir Vapnik. 1996. Support vector regression machines. In *Advances in Neural Information Processing Systems* 9 (1996).
- [16] Rolf Enzler, Tobias Jeger, Didier Cottet, and Gerhard Tröster. 2000. High-level area and performance estimation of hardware building blocks on FPGAs. In *Proceedings of the International Workshop on Field Programmable Logic and Applications*. 525–534.
- [17] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. 2000. Additive logistic regression: A statistical view of boosting (with discussion and a rejoinder by the authors). *Annals of Statistics* 28, 2 (2000), 337–407.
- [18] Yuko Hara, Hiroyuki Tomiyama, Shinya Honda, Hiroaki Takada, and Katsuya Ishii. 2008. CHStone: A benchmark program suite for practical C-based high-level synthesis. In *Proceedings of the 2008 IEEE International Symposium on Circuits and Systems*. IEEE, Los Alamitos, CA, 1192–1195.
- [19] Jonathan Babb, Russell Tessier, and Anant Agarwal. 1993. Virtual wires: Overcoming pin limitations in FPGA-based logic emulators. In *[1993] Proceedings IEEE Workshop on FPGAs for Custom Computing Machines*. IEEE, 142–151.
- [20] K. Hornik, M. Stinchcombe, and H. White. 1989. Multilayer feedforward networks are universal approximators. *Neural Networks* 2, 5 (1989), 359–366.
- [21] Guyue Huang, Jingbo Hu, Yifan He, Jialong Liu, Mingyuan Ma, Zhaoyang Shen, Juejian Wu, et al. 2021. Machine learning for electronic design automation: A survey. *ACM Transactions on Design Automation of Electronic Systems* 26, 5 (2021), Article 40, 46 pages.
- [22] William N. N. Hung and Richard Sun. 2018. Challenges in large FPGA-based logic emulation systems. In *Proceedings of the 2018 International Symposium on Physical Design*. 26–33.
- [23] Benzhen Li, Zhongdong Qi, Zhengguang Tang, Xiyi He, and Hailong You. 2021. High quality hypergraph partitioning for logic emulation. *Integration* 83 (2021), 67–76.
- [24] Mariem Makni, Mouna Baklouti, Smail Niar, and Mohamed Abid. 2017. Hardware resource estimation for heterogeneous FPGA-based SoCs. In *Proceedings of the 32nd ACM Symposium on Applied Computing (SAC’17)*.
- [25] H. M. Makrani, F. Farahmand, H. Sayadi, S. Bondi, S. Manoj, H. Homayoun, and S. Rafatirad. 2019. Pyramid: Machine learning framework to estimate the optimal timing and resource usage of a high-level synthesis design. In *Proceedings of the 2019 29th International Conference on Field Programmable Logic and Applications (FPL’19)*.
- [26] Roel Meeuws, S. Arash Ostadzadeh, Carlo Galuzzi, Vlad Mihai Sima, Razvan Nane, and Koen Bertels. 2013. Quipu: A statistical model for predicting hardware resources. *ACM Transactions on Reconfigurable Technology and Systems* 6, 1 (2013), 1–25.
- [27] Anshuman Nayak, Malay Haldar, Alok Choudhary, and Prithviraj Banerjee. 2002. Accurate area and delay estimators for FPGAs. In *Proceedings of the 2002 Design, Automation, and Test in Europe Conference and Exhibition (DATE’02)*. IEEE, Los Alamitos, CA, 862–869.
- [28] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [29] B. Reagen, R. Adolf, Y. S. Shao, G. Y. Wei, and D. Brooks. 2014. MachSuite: Benchmarks for accelerator design and customized architectures. In *Proceedings of the IEEE International Symposium on Workload Characterization*. 110–119.
- [30] Paul Schumacher and Pradip Jha. 2008. Fast and accurate resource estimation of RTL-based designs targeting FPGAs. In *Proceedings of the 2008 International Conference on Field Programmable Logic and Applications*. IEEE, Los Alamitos, CA, 59–64.
- [31] M. Shahshahani and D. Bhatia. 2021. Resource and performance estimation for CNN models using machine learning. In *Proceedings of the 2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI’21)*.
- [32] Changchun Shi, James Hwang, Scott Mcmillan, Ann Root, and Vinay Singh. 2004. A system level resource estimation tool for FPGAs. In *Proceedings of the International Conference on Field Programmable Logic and Applications (FPL’04)*. 424–433.
- [33] Ecenur Ustun, Shaojie Xiang, Jinny Gui, Cunxi Yu, and Zhiru Zhang. 2019. LAMDA: Learning-assisted multi-stage autotuning for FPGA design closure. In *Proceedings of the 2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM’19)*. IEEE, Los Alamitos, CA, 74–77.

Received 14 February 2022; revised 15 July 2022; accepted 24 July 2022